

基础算法专题

枚举 • 贪心 • 二分/三分 • 前缀和与差分 • 排序

圣文卓

2026 年 5 月 8 日

目录

1	枚举	3
1.1	子集枚举（二进制枚举）	3
1.2	全排列枚举（DFS 回溯）	4
1.3	组合枚举	5
1.4	四平方和（枚举 + 哈希优化）	5
1.5	百钱百鸡（经典枚举优化）	7
1.6	回文日期	7
2	贪心	9
2.1	区间选点（最少点覆盖所有区间）	9
2.2	最大不相交区间数量	10
2.3	区间覆盖（最小区间数覆盖线段）	11
2.4	合并果子（哈夫曼编码）	12
2.5	排队接水（最小等待时间）	13
2.6	货仓选址（绝对值不等式）	14
3	二分 / 三分	15
3.1	整数二分查找（精确匹配）	15
3.2	二分查找左边界（第一个 $\geq x$ 的位置）	16
3.3	二分查找右边界（最后一个 $\leq x$ 的位置）	16
3.4	浮点数二分（求平方根/立方根）	17
3.5	二分答案——最小化最大值	18
3.6	三分查找（单峰函数极值）	19
3.7	二分查找旋转数组的最小值	20

4	前缀和与差分	22
4.1	一维前缀和——区间和查询	22
4.2	二维前缀和——子矩阵和查询	23
4.3	一维差分——区间增减操作	24
4.4	二维差分——子矩阵增减操作	24
4.5	前缀和——区间内特定值出现次数	25
5	排序	27
5.1	快速排序	27
5.2	归并排序	28
5.3	归并排序求逆序对数量	29
5.4	快速选择（求第 k 小的数）	30
5.5	计数排序	31
5.6	基数排序	32
	附录：算法对比总结	34

1 枚举

枚举（Enumeration）是最基础的算法思想：遍历解空间中的所有候选解，逐一检验是否满足条件。适用于数据规模较小、无法或不需要推导数学公式的情形。核心在于控制枚举范围和剪枝优化，避免不必要的循环。

1.1 子集枚举（二进制枚举）

适用场景：从 n 个元素构成的集合中，枚举所有可能的子集（共 2^n 个），每个元素选或不选，适合 $n \leq 20$ 。

题目：给定一个含有 n 个整数的数组，输出所有子集的元素和。

```
1 #include <iostream>
2 #include <vector>
3 using namespace std;
4
5 int main() {
6     int n;                // n: 数组元素个数
7     cin >> n;
8     vector<int> a(n);      // a[i]: 第i个元素的值
9     for (int i = 0; i < n; i++) cin >> a[i];
10    // 枚举 [0, 2^n - 1] 之间的每一个二进制掩码
11    int total = 1 << n;    // total = 2^n, 子集总数
12    for (int mask = 0; mask < total; mask++) {
13        int sum = 0;      // sum: 当前子集的元素和
14        // 检查 mask 的每一位，第 j 位为 1 表示选择 a[j]
15        for (int j = 0; j < n; j++) {
16            if (mask & (1 << j)) { // 判断 mask 的第 j 位是否为 1
17                sum += a[j];
18            }
19        }
20        cout << "subset " << mask << " sum = " << sum << endl;
21    }
22    return 0;
23 }
```

Listing 1: 二进制枚举所有子集

1.2 全排列枚举（DFS 回溯）

适用场景：生成 $1 \sim n$ 的所有排列，或对数组元素进行全排列后暴力验证，适合 $n \leq 10$ 。

题目：输出 $1 \sim n$ 的所有全排列。

```
1  #include <iostream>
2  #include <vector>
3  using namespace std;
4
5  int n;                // n: 排列长度
6  vector<int> path;      // path: 当前排列路径
7  vector<bool> used;     // used[i]: 数字 i 是否已被使用
8
9  // 原理：深度优先搜索，每次从未使用的数字中选一个加入当前路径
10 void dfs() {
11     if ((int)path.size() == n) { // 找到一个完整排列
12         for (int x : path) cout << x << " ";
13         cout << endl;
14         return;
15     }
16     for (int i = 1; i <= n; i++) { // 按字典序尝试每个数字
17         if (used[i]) continue; // 已使用则跳过（剪枝）
18         used[i] = true;        // 标记使用
19         path.push_back(i);      // 加入路径
20         dfs();                  // 递归搜索下一层
21         path.pop_back();        // 回溯：撤销选择
22         used[i] = false;       // 回溯：恢复未使用状态
23     }
24 }
25
26 int main() {
27     cin >> n;
28     used.assign(n + 1, false);
29     dfs();
30     return 0;
31 }
```

Listing 2: DFS 回溯生成全排列

1.3 组合枚举

适用场景：从 n 个元素中选出恰好 k 个，需要枚举所有组合方案，适合 $n \leq 25$ 。

题目：从 $1 \sim n$ 中选出 k 个数，按字典序输出所有组合。

```
1  #include <iostream>
2  #include <vector>
3  using namespace std;
4
5  int n, k;                // n: 总数, k: 选取个数
6  vector<int> comb;        // comb: 当前组合
7
8  // 原理: 每次从 start 开始选数, 保证组合内部递增, 避免重复
9  void dfs(int start) {
10     if ((int)comb.size() == k) { // 已选够 k 个, 输出
11         for (int x : comb) cout << x << " ";
12         cout << endl;
13         return;
14     }
15     // 剪枝: 剩余可选数字数量不足时提前返回
16     // 还需要选 k-comb.size() 个, 最多能从 n 选到 start
17     for (int i = start; i <= n - (k - (int)comb.size()) + 1; i++) {
18         comb.push_back(i);
19         dfs(i + 1);          // 下一层从 i+1 开始, 保证递增
20         comb.pop_back();     // 回溯
21     }
22 }
23
24 int main() {
25     cin >> n >> k;
26     dfs(1);
27     return 0;
28 }
```

Listing 3: DFS 枚举组合（字典序）

1.4 四平方和（枚举 + 哈希优化）

适用场景：将枚举从 $O(n^3)$ 或 $O(n^4)$ 优化到 $O(n^2)$ ，通过预处理一部分结果并用哈希表或数组存储，将“枚举前半 + 查找后半”的思想用于降低复杂度。

题目：每个正整数都可以表示为至多 4 个正整数的平方和。给定 $N \leq 5 \times 10^6$ ，求字典序最小的 a, b, c, d 使得 $a^2 + b^2 + c^2 + d^2 = N$ 且 $a \leq b \leq c \leq d$ 。

```

1  #include <iostream>
2  #include <cmath>
3  #include <cstring>
4  using namespace std;
5
6  const int MAXN = 5000010;    // MAXN: 根据 N 上限设定
7  int h[MAXN];                // h[x]: 当两数平方和为 x 时, 较小的那个数
8
9  int main() {
10     int N;                    // N: 目标正整数
11     cin >> N;
12     memset(h, -1, sizeof(h)); // -1 表示未出现过
13     // 预处理: 枚举后两个数的平方和 c^2 + d^2
14     for (int c = 0; c * c <= N; c++) {
15         for (int d = c; c * c + d * d <= N; d++) { // d 从 c 开始保证 c<=d
16             int sum = c * c + d * d;
17             if (h[sum] == -1) { // 只记录第一次出现 (保证字典序最小)
18                 h[sum] = c;    // 存储较小的数 c, 较大数 d 可由 sum-c^2 算出
19             }
20         }
21     }
22     // 枚举前两个数 a^2 + b^2, 查表匹配后两个数
23     for (int a = 0; a * a <= N; a++) {
24         for (int b = a; a * a + b * b <= N; b++) {
25             int rest = N - a * a - b * b; // rest: 剩余需要凑的值
26             if (rest < 0) break;
27             if (h[rest] == -1) continue; // 无法凑出剩余值
28             int c = h[rest];            // c: 查表得到的第三个数
29             int d = sqrt(rest - c * c); // d: 由 rest - c^2 开方得到
30             cout << a << " " << b << " " << c << " " << d << endl;
31             return 0;                    // 字典序最小, 找到即退出
32         }
33     }
34     return 0;
35 }

```

Listing 4: 四平方和——枚举前两项 + 哈希查表

1.5 百钱百鸡（经典枚举优化）

适用场景：多元一次不定方程求整数解，通过减少变量个数将多重循环降为单重循环，适用于小数据范围的约束满足问题。

题目：公鸡 5 元一只，母鸡 3 元一只，小鸡 1 元三只。用 100 元买 100 只鸡，求所有购买方案。

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      // 原理：设公鸡 x 只，母鸡 y 只，小鸡 z 只
6      //  $x + y + z = 100$ ,  $5x + 3y + z/3 = 100$  且 z 是 3 的倍数
7      // 由两式消去 z，得  $7x + 4y = 100$ ，枚举 x 即可确定 y, z
8      for (int x = 0; x <= 20; x++) { // x: 公鸡数,  $5x \leq 100$  故  $x \leq 20$ 
9          int y = (100 - 7 * x) / 4; // y: 母鸡数，由方程解出
10         int z = 100 - x - y;      // z: 小鸡数
11         // 验证方程是否成立且 y, z 为非负整数
12         if (y >= 0 && z >= 0 && z % 3 == 0 && 5*x + 3*y + z/3 == 100) {
13             cout << "cock=" << x << " hen=" << y << " chick=" << z << endl;
14         }
15     }
16     return 0;
17 }
```

Listing 5: 百钱百鸡——消元降维枚举

1.6 回文日期

适用场景：枚举日期或回文数，通过生成而非逐一判定的方式缩小枚举范围。适用于日期/数字回文类问题。

题目：给定起始日期和终止日期（8 位数字），求在此范围内的回文日期个数。

```
1  #include <iostream>
2  using namespace std;
3
4  int days[13] = {0, 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
5  // days[m]: 平年第 m 月的天数
6
7  // 判断是否为闰年
8  bool isLeap(int y) {
9      return (y % 4 == 0 && y % 100 != 0) || (y % 400 == 0);
```

```
10 }
11
12 // 判断日期是否合法
13 bool isValid(int y, int m, int d) {
14     if (m < 1 || m > 12) return false; // 月份非法
15     int maxd = days[m];                // maxd: 当月最大天数
16     if (m == 2 && isLeap(y)) maxd = 29; // 闰年二月有 29 天
17     return d >= 1 && d <= maxd;
18 }
19
20 int main() {
21     int start, end;                    // start: 起始日期, end: 终止日期
22     cin >> start >> end;
23     int ans = 0;                       // ans: 回文日期个数
24     // 原理: 枚举年份前四位, 翻转拼接得到完整回文日期, 再检查合法性
25     for (int y = start / 10000; y <= end / 10000; y++) { // y: 年份
26         int rev = 0;                   // rev: y 的翻转数
27         int tmp = y;
28         for (int k = 0; k < 4; k++) {
29             rev = rev * 10 + tmp % 10;
30             tmp /= 10;
31         }
32         int m = rev / 100;              // m: 月份 (翻转后的前两位)
33         int d = rev % 100;              // d: 日期 (翻转后的后两位)
34         int date = y * 10000 + rev; // date: 完整回文日期
35         if (date >= start && date <= end && isValid(y, m, d)) {
36             ans++;
37         }
38     }
39     cout << ans << endl;
40     return 0;
41 }
```

Listing 6: 回文日期——枚举前四位生成回文

2 贪心

贪心算法（Greedy）在每一步选择中都采取当前状态下最优的选择，期望最终全局最优。使用贪心的前提是问题具有**贪心选择性质**（局部最优能推出全局最优）和**最优子结构**。

2.1 区间选点（最少点覆盖所有区间）

适用场景：给定多个区间 $[l_i, r_i]$ ，在数轴上放置最少的点使得每个区间内至少有一个点。典型应用：雷达覆盖、传感器部署。

题目：给定 N 个闭区间，求最少需要选多少个点使得每个区间内至少包含一个选出的点。

```
1  #include <iostream>
2  #include <algorithm>
3  using namespace std;
4
5  const int MAXN = 100005;      // MAXN: 最大区间数
6  struct Range {
7      int l, r;                  // l: 左端点, r: 右端点
8      // 按右端点升序排序: 右端点越靠左, 留给后面区间的空间越大
9      bool operator<(const Range& other) const {
10         return r < other.r;
11     }
12 } ranges[MAXN];
13
14 int main() {
15     int n;                      // n: 区间个数
16     cin >> n;
17     for (int i = 0; i < n; i++) cin >> ranges[i].l >> ranges[i].r;
18     sort(ranges, ranges + n);   // 按右端点排序
19     int ans = 0;                 // ans: 所选点数
20     int last = -2e9;             // last: 上一个选点的坐标 (初始为极小值)
21     // 原理: 每次选当前区间右端点, 能尽可能多地覆盖后续区间
22     for (int i = 0; i < n; i++) {
23         if (ranges[i].l > last) { // 当前区间未被上一个点覆盖
24             ans++;                // 新增一个点
25             last = ranges[i].r;   // 点选在当前区间右端点
26         }
27     }
```

```
28     cout << ans << endl;
29     return 0;
30 }
```

Listing 7: 贪心——按右端点排序，每次选右端点

2.2 最大不相交区间数量

适用场景：从一组区间中选出尽可能多的互不重叠的区间，如活动安排、课程排课、会议室预定。

题目：给定 N 个区间，选出最多的区间使得它们两两不相交。

```
1  #include <iostream>
2  #include <algorithm>
3  using namespace std;
4
5  const int MAXN = 100005;
6  struct Range {
7      int l, r;                // l: 左端点, r: 右端点
8      bool operator<(const Range& other) const {
9          return r < other.r;  // 按右端点升序
10     }
11 } ranges[MAXN];
12
13 int main() {
14     int n;                    // n: 区间个数
15     cin >> n;
16     for (int i = 0; i < n; i++) cin >> ranges[i].l >> ranges[i].r;
17     sort(ranges, ranges + n);
18     int ans = 0;              // ans: 最大不相交区间数
19     int last = -2e9;          // last: 上一个选中区间的右端点
20     // 原理: 每次选最早结束的区间, 给后续区间留下最大空间
21     for (int i = 0; i < n; i++) {
22         if (ranges[i].l > last) { // 当前区间不与上一个选中区间重叠
23             ans++;
24             last = ranges[i].r;
25         }
26     }
27     cout << ans << endl;
28     return 0;
29 }
```

Listing 8: 贪心——同类按右端点排序，每次选最早结束的

2.3 区间覆盖（最小区间数覆盖线段）

适用场景：用最少的给定区间覆盖一条指定线段，如铺设管道、信号覆盖、灌溉问题。

题目：给定 N 个区间和一个目标线段 $[start, end]$ ，求最少需要多少个区间完全覆盖目标线段。

```
1  #include <iostream>
2  #include <algorithm>
3  using namespace std;
4
5  const int MAXN = 100005;
6  struct Range {
7      int l, r;
8      bool operator<(const Range& other) const {
9          return l < other.l;    // 按左端点升序，方便从左到右扫描
10     }
11 } ranges[MAXN];
12
13 int main() {
14     int st, ed;                // st: 目标线段起点, ed: 目标线段终点
15     cin >> st >> ed;
16     int n;                    // n: 可选区间个数
17     cin >> n;
18     for (int i = 0; i < n; i++) cin >> ranges[i].l >> ranges[i].r;
19     sort(ranges, ranges + n);
20     int ans = 0;               // ans: 最少所需区间数
21     int cur = st;              // cur: 当前已覆盖到的位置（缺口左端）
22     int i = 0;                 // i: 当前遍历到的区间下标
23     // 原理：在左端点 <= cur 的所有区间中，选右端点最大的那个
24     while (cur < ed) {
25         int maxr = -2e9;       // maxr: 能覆盖到的最远右端点
26         while (i < n && ranges[i].l <= cur) {
27             maxr = max(maxr, ranges[i].r);
28             i++;
29         }
30         if (maxr <= cur) {      // 无法继续扩展，覆盖失败
```

```

31         cout << -1 << endl;
32         return 0;
33     }
34     ans++;
35     cur = maxr;           // 更新已覆盖位置
36 }
37 cout << ans << endl;
38 return 0;
39 }

```

Listing 9: 贪心——每次选能覆盖当前缺口且右端点最远的区间

2.4 合并果子（哈夫曼编码）

适用场景：将若干堆物品两两合并，每次合并的代价为两堆之和，求最小总代价。适用于数据压缩（哈夫曼树）、最优合并问题。

题目：有 n 堆果子，每次合并任意两堆，代价为两堆重量之和。求将所有堆合并成一堆的最小代价。

```

1  #include <iostream>
2  #include <queue>
3  using namespace std;
4
5  int main() {
6      int n;           // n: 果子堆数
7      cin >> n;
8      priority_queue<int, vector<int>, greater<int>>> heap;
9      // heap: 小根堆，每次能 O(log n) 取出最小值
10     for (int i = 0; i < n; i++) {
11         int x;         // x: 第 i 堆的重量
12         cin >> x;
13         heap.push(x);
14     }
15     int ans = 0;       // ans: 最小总代价
16     // 原理：哈夫曼算法——每次合并最小的两堆，合并后放回堆中
17     while (heap.size() > 1) {
18         int a = heap.top(); heap.pop(); // a: 当前最小堆
19         int b = heap.top(); heap.pop(); // b: 当前次小堆
20         int cost = a + b;             // cost: 合并 a 和 b 的代价
21         ans += cost;                  // 累加总代价
22         heap.push(cost);              // 新堆放回堆中

```

```
23     }
24     cout << ans << endl;
25     return 0;
26 }
```

Listing 10: 贪心——小根堆每次取最小的两堆合并

2.5 排队接水（最小等待时间）

适用场景：多人排队接受服务，每人服务耗时不同，调整顺序使总等待时间最小。适用于任务调度、客服排队。

题目： n 个人排队接水，第 i 人需要 t_i 时间。求一种排队顺序使所有人的等待时间之和最小。

```
1  #include <iostream>
2  #include <algorithm>
3  using namespace std;
4
5  const int MAXN = 1005;
6  struct Person {
7      int id;                // id: 人员编号
8      int t;                // t: 接水所需时间
9      bool operator<(const Person& other) const {
10         return t < other.t; // 按时间升序，短作业优先
11     }
12 } p[MAXN];
13
14 int main() {
15     int n;                // n: 人数
16     cin >> n;
17     for (int i = 0; i < n; i++) {
18         p[i].id = i + 1;
19         cin >> p[i].t;
20     }
21     sort(p, p + n);        // 短作业优先排序
22     long long total = 0;    // total: 总等待时间
23     long long cur = 0;      // cur: 当前累计接水时间
24     // 原理：第 i 个人的等待时间 = 前面所有人的接水时间之和
25     for (int i = 0; i < n; i++) {
26         total += cur;        // cur 就是此人的等待时间
27         cur += p[i].t;       // 加上此人接水时间，传给下一个人
28     }
```

```

28     }
29     // 输出顺序
30     for (int i = 0; i < n; i++) cout << p[i].id << " ";
31     cout << endl;
32     printf("%.2f\n", (double)total / n); // 平均等待时间
33     return 0;
34 }

```

Listing 11: 贪心——按服务时间升序排队，短作业优先

2.6 货仓选址（绝对值不等式）

适用场景：在一条直线上选择一点，使得到所有给定点的距离之和最小。适用于物流中心选址、中位数优化问题。

题目：数轴上有 N 个点，选择一个点使得到所有点的距离之和最小，求最小距离和。

```

1  #include <iostream>
2  #include <algorithm>
3  using namespace std;
4
5  const int MAXN = 100005;
6  int a[MAXN];           // a[i]: 第 i 个点的坐标
7
8  int main() {
9      int n;             // n: 点数
10     cin >> n;
11     for (int i = 0; i < n; i++) cin >> a[i];
12     sort(a, a + n);     // 排序后取中位数
13     int mid = a[n / 2];  // mid: 中位数坐标
14     long long ans = 0;   // ans: 最小距离和
15     // 原理：绝对值函数 |x-a| 的最优解 x 是 a 的中位数
16     for (int i = 0; i < n; i++) {
17         ans += abs(a[i] - mid);
18     }
19     cout << ans << endl;
20     return 0;
21 }

```

Listing 12: 贪心——选中位数，利用绝对值不等式

3 二分 / 三分

二分查找 (Binary Search) 在单调函数或有序序列上每次排除一半的搜索空间，复杂度 $O(\log n)$ 。三分查找用于单峰/单谷函数求极值。二分答案则将最优化问题转化为判定问题。

3.1 整数二分查找 (精确匹配)

适用场景：在有序数组中查找目标值是否存在，返回其下标。

题目：给定一个升序数组，判断目标值 x 是否存在。

```
1  #include <iostream>
2  using namespace std;
3
4  const int MAXN = 100005;
5  int a[MAXN];           // a[]: 升序数组
6
7  int main() {
8      int n, x;           // n: 数组长度, x: 目标值
9      cin >> n >> x;
10     for (int i = 0; i < n; i++) cin >> a[i];
11     int l = 0, r = n - 1; // l: 左边界, r: 右边界 (闭区间)
12     int ans = -1;         // ans: 目标下标, -1 表示未找到
13     // 原理: 每次比较中间值, 根据大小关系舍弃一半区间
14     while (l <= r) {
15         int mid = l + (r - l) / 2; // mid: 中间下标 (防溢出写法)
16         if (a[mid] == x) {
17             ans = mid;
18             break;
19         } else if (a[mid] < x) {
20             l = mid + 1;           // x 在右半部分
21         } else {
22             r = mid - 1;           // x 在左半部分
23         }
24     }
25     cout << ans << endl;
26     return 0;
27 }
```

Listing 13: 整数二分——查找目标值

3.2 二分查找左边界（第一个 $\geq x$ 的位置）

适用场景：在有序数组中查找第一个大于等于目标值的位置。典型应用：有序数组插入位置、找第一个满足条件的元素。

题目：给定升序数组，求第一个大于等于 x 的元素下标，不存在则输出 n 。

```
1  #include <iostream>
2  using namespace std;
3
4  const int MAXN = 100005;
5  int a[MAXN];
6
7  int main() {
8      int n, x;
9      cin >> n >> x;
10     for (int i = 0; i < n; i++) cin >> a[i];
11     int l = 0, r = n;          // r = n 表示答案可能是 n（不存在的情况）
12     // 原理：维护答案区间 [l, r)，每次将区间二分
13     while (l < r) {
14         int mid = l + (r - l) / 2;
15         if (a[mid] >= x) {
16             r = mid;           // mid 满足条件，收缩右边界
17         } else {
18             l = mid + 1;       // mid 不满足，收缩左边界
19         }
20     }
21     cout << l << endl;        // l 即为第一个 >= x 的下标
22     return 0;
23 }
```

Listing 14: 二分查找——第一个大于等于 x 的位置 (lower_bound)

3.3 二分查找右边界（最后一个 $\leq x$ 的位置）

适用场景：在有序数组中查找最后一个小于等于目标值的位置。适用于查找某个元素最后一次出现的位置。

```
1  #include <iostream>
2  using namespace std;
3
4  const int MAXN = 100005;
5  int a[MAXN];
```

```
6
7 int main() {
8     int n, x;
9     cin >> n >> x;
10    for (int i = 0; i < n; i++) cin >> a[i];
11    int l = 0, r = n - 1;    // 闭区间 [l, r]
12    int ans = -1;           // ans: 最后一个 <= x 的下标
13    while (l <= r) {
14        int mid = l + (r - l) / 2;
15        if (a[mid] <= x) {
16            ans = mid;       // 记录当前满足条件的下标
17            l = mid + 1;     // 尝试找更靠后的位置
18        } else {
19            r = mid - 1;
20        }
21    }
22    cout << ans << endl;
23    return 0;
24 }
```

Listing 15: 二分查找——最后一个小于等于 x 的位置

3.4 浮点数二分（求平方根/立方根）

适用场景：在实数范围内利用二分逼近解，如求方程根、开方、最大化平均值等。

题目：给定一个正实数 x ，求其平方根（精确到 10^{-6} ）。

```
1 #include <iostream>
2 #include <iomanip>
3 #include <cmath>
4 using namespace std;
5
6 int main() {
7     double x;           // x: 被开方数
8     cin >> x;
9     double l = 0, r = max(1.0, x); // l: 左边界, r: 右边界 (注意 x<1 时平方根 > x)
10    const double eps = 1e-8; // eps: 精度要求
11    // 原理: 在实数区间上二分, 直至区间长度小于精度
12    while (r - l > eps) {
13        double mid = (l + r) / 2; // mid: 区间中点
```

```

14     if (mid * mid >= x) {
15         r = mid;           // mid 的平方太大，平方根在左侧
16     } else {
17         l = mid;           // mid 的平方太小，平方根在右侧
18     }
19 }
20 cout << fixed << setprecision(6) << l << endl;
21 return 0;
22 }
```

Listing 16: 浮点数二分——求平方根

3.5 二分答案——最小化最大值

适用场景：答案具有**单调性**：如果某个值可行，则更大的值一定可行。将”求最小/最大的最优值”转化为”判断某个值是否可行”的判定问题。

题目：将 n 个正整数分成连续的 m 段，使得各段和的最大值最小，求这个最小值。

```

1  #include <iostream>
2  using namespace std;
3
4  const int MAXN = 100005;
5  int a[MAXN];           // a[i]: 第 i 个数
6  int n, m;              // n: 数字个数, m: 要分成的段数
7
8  // 判断在每段和不超过 limit 的情况下能否分成 <= m 段
9  bool check(long long limit) {
10     int cnt = 1;         // cnt: 段数 (至少为 1)
11     long long sum = 0;    // sum: 当前段的累加和
12     for (int i = 0; i < n; i++) {
13         if (a[i] > limit) return false; // 单个元素大于限制, 不可能
14         if (sum + a[i] > limit) { // 当前段已满
15             cnt++;              // 新开一段
16             sum = a[i];         // 新段的第一个元素
17         } else {
18             sum += a[i];
19         }
20     }
21     return cnt <= m;        // 段数是否在允许范围内
22 }
23
```

```

24 int main() {
25     cin >> n >> m;
26     long long l = 0, r = 0;    // l: 二分左界（最大单元素），r: 二分右界（总和）
27     for (int i = 0; i < n; i++) {
28         cin >> a[i];
29         l = max(l, (long long)a[i]);
30         r += a[i];
31     }
32     // 原理: limit 越大越容易满足，答案单调，二分查找最小的可行 limit
33     while (l < r) {
34         long long mid = l + (r - l) / 2;
35         if (check(mid)) {
36             r = mid;          // mid 可行，尝试更小的值
37         } else {
38             l = mid + 1;      // mid 不可行，需要更大的值
39         }
40     }
41     cout << l << endl;
42     return 0;
43 }

```

Listing 17: 二分答案——最小化各段和的最大值

3.6 三分查找（单峰函数极值）

适用场景：求单峰函数（先增后减）的极大值或单谷函数（先减后增）的极小值，适用于凸优化问题。

题目：给定一个先增后减的数组，求其最大值。

```

1  #include <iostream>
2  using namespace std;
3
4  const int MAXN = 100005;
5  int a[MAXN];          // a[]: 单峰数组（先严格递增再严格递减）
6
7  int main() {
8      int n;             // n: 数组长度
9      cin >> n;
10     for (int i = 0; i < n; i++) cin >> a[i];
11     int l = 0, r = n - 1; // l: 左边界, r: 右边界
12     // 原理: 取两个三分点 m1, m2, 比较大小, 每次舍弃单调性不符的那 1/3 区间

```

```

13 while (r - l > 2) {           // 区间足够大时才三分
14     int m1 = l + (r - l) / 3; // m1: 左三分点
15     int m2 = r - (r - l) / 3; // m2: 右三分点
16     if (a[m1] < a[m2]) {
17         l = m1;               // 最大值在 m1 右侧
18     } else {
19         r = m2;               // 最大值在 m2 左侧
20     }
21 }
22 // 小区间内直接扫描
23 int ans = a[l];
24 for (int i = l + 1; i <= r; i++) ans = max(ans, a[i]);
25 cout << ans << endl;
26 return 0;
27 }

```

Listing 18: 三分查找——求单峰数组最大值

3.7 二分查找旋转数组的最小值

适用场景：有序数组经过旋转（前面若干元素移到末尾）后查找最小值或目标值。利用部分有序的特性仍可二分。

题目：给定一个升序数组经过一次旋转后的数组（元素互不相同），求其最小值。

```

1  #include <iostream>
2  using namespace std;
3
4  const int MAXN = 100005;
5  int a[MAXN];           // a[]: 旋转后的数组（原数组升序无重复）
6
7  int main() {
8      int n;              // n: 数组长度
9      cin >> n;
10     for (int i = 0; i < n; i++) cin >> a[i];
11     int l = 0, r = n - 1;
12     // 原理：将 a[mid] 与 a[r] 比较，判断最小值在左半还是右半
13     while (l < r) {
14         int mid = l + (r - l) / 2;
15         if (a[mid] < a[r]) {
16             r = mid;      // 右半有序，最小值在左半（含 mid）
17         } else {

```

```
18         l = mid + 1;           // 左半有序，最小值在右半
19     }
20 }
21 cout << a[l] << endl;        // l 即为最小值下标
22 return 0;
23 }
```

Listing 19: 二分查找——旋转有序数组中的最小值

4 前缀和与差分

前缀和 (Prefix Sum) 将区间求和从 $O(n)$ 降至 $O(1)$; 差分 (Difference) 将区间增减从 $O(n)$ 降至 $O(1)$ 。两者互为逆运算, 是处理批量区间操作的核心工具。

4.1 一维前缀和——区间和查询

适用场景: 静态数组上频繁查询任意区间 $[L, R]$ 的元素和, 预处理 $O(n)$, 单次查询 $O(1)$ 。

题目: 给定 n 个整数和 q 个查询, 每次查询区间 $[L, R]$ 的元素和。

```
1  #include <iostream>
2  using namespace std;
3
4  const int MAXN = 100005;
5  int a[MAXN];           // a[i]: 原数组第 i 个元素 (1-indexed)
6  long long pre[MAXN];   // pre[i]: 前缀和, pre[i] = a[1] + a[2] + ... +
                          // a[i]
7
8  int main() {
9      int n, q;           // n: 数组长度, q: 查询次数
10     cin >> n >> q;
11     for (int i = 1; i <= n; i++) cin >> a[i];
12     // 预处理前缀和: pre[i] = pre[i-1] + a[i]
13     for (int i = 1; i <= n; i++) {
14         pre[i] = pre[i-1] + a[i];
15     }
16     while (q--) {
17         int L, R;        // L: 查询左端点, R: 查询右端点
18         cin >> L >> R;
19         // 原理: 区间和 = 前缀和[R] - 前缀和[L-1]
20         cout << pre[R] - pre[L-1] << endl;
21     }
22     return 0;
23 }
```

Listing 20: 一维前缀和—— $O(1)$ 区间和查询

4.2 二维前缀和——子矩阵和查询

适用场景：静态矩阵上频繁查询子矩阵的元素和，如激光炸弹、图像积分图、子矩阵求和。预处理 $O(nm)$ ，查询 $O(1)$ 。

题目：给定 $n \times m$ 矩阵和 q 个查询，每次求 (x_1, y_1) 到 (x_2, y_2) 子矩阵的元素和。

```

1  #include <iostream>
2  using namespace std;
3
4  const int MAXN = 1005;
5  int a[MAXN][MAXN];           // a[i][j]: 原矩阵第 i 行第 j 列元素
6  long long pre[MAXN][MAXN];   // pre[i][j]: (1,1) 到 (i,j) 的矩阵和
7
8  int main() {
9      int n, m, q;              // n: 行数, m: 列数, q: 查询次数
10     cin >> n >> m >> q;
11     for (int i = 1; i <= n; i++)
12         for (int j = 1; j <= m; j++)
13             cin >> a[i][j];
14     // 预处理二维前缀和
15     // 原理: pre[i][j] = pre[i-1][j] + pre[i][j-1] - pre[i-1][j-1] + a[i][j]
16     for (int i = 1; i <= n; i++) {
17         for (int j = 1; j <= m; j++) {
18             pre[i][j] = pre[i-1][j] + pre[i][j-1] - pre[i-1][j-1] + a[i][j];
19         }
20     }
21     while (q--) {
22         int x1, y1, x2, y2;    // (x1,y1): 左上角, (x2,y2): 右下角
23         cin >> x1 >> y1 >> x2 >> y2;
24         // 原理: 子矩阵和 = 右下角前缀和 - 左 - 上 + 左上 (容斥原理)
25         long long ans = pre[x2][y2] - pre[x1-1][y2] - pre[x2][y1-1] + pre[x1-1][y1-1];
26         cout << ans << endl;
27     }
28     return 0;
29 }

```

Listing 21: 二维前缀和—— $O(1)$ 子矩阵和查询

4.3 一维差分——区间增减操作

适用场景：对一个数组进行多次区间加减操作，最后查询最终数组。差分将每次区间修改从 $O(n)$ 降到 $O(1)$ 。

题目：初始全零数组，进行 m 次操作：对 $[L, R]$ 区间每个元素加 c 。输出最终数组。

```

1  #include <iostream>
2  using namespace std;
3
4  const int MAXN = 100005;
5  int diff[MAXN];           // diff[i]: 差分数组, diff[i] = a[i] - a[i-1]
6
7  int main() {
8      int n, m;             // n: 数组长度, m: 操作次数
9      cin >> n >> m;
10     // diff 初始全 0 (对应原数组全 0)
11     // 原理: 对 [L,R] 加 c, 等价于 diff[L] += c, diff[R+1] -= c
12     while (m--) {
13         int L, R, c;       // L,R: 操作区间, c: 增加值
14         cin >> L >> R >> c;
15         diff[L] += c;      // 从 L 开始影响生效
16         diff[R + 1] -= c;  // 从 R+1 开始影响消除
17     }
18     // 对差分数组做前缀和还原原数组
19     int cur = 0;           // cur: 当前位置的前缀和即 a[i]
20     for (int i = 1; i <= n; i++) {
21         cur += diff[i];    // cur = a[i]
22         cout << cur << " ";
23     }
24     cout << endl;
25     return 0;
26 }
```

Listing 22: 一维差分—— $O(1)$ 区间增减

4.4 二维差分——子矩阵增减操作

适用场景：对矩阵进行多次子矩阵批量加减，最后输出最终矩阵。典型应用：二维区间修改、棋盘覆盖。

题目：初始全零 $n \times m$ 矩阵， q 次操作：对 (x_1, y_1) 到 (x_2, y_2) 子矩阵每个元素加 c 。输出最终矩阵。

```

1  #include <iostream>
2  using namespace std;
3
4  const int MAXN = 1005;
5  int diff[MAXN][MAXN];           // diff[i][j]: 二维差分数组
6
7  int main() {
8      int n, m, q;                // n: 行数, m: 列数, q: 操作次数
9      cin >> n >> m >> q;
10     // 原理: 子矩阵加 c 转化为四个角上的差分操作 (二维容斥)
11     // diff[x1][y1] += c, diff[x2+1][y1] -= c,
12     // diff[x1][y2+1] -= c, diff[x2+1][y2+1] += c
13     while (q--) {
14         int x1, y1, x2, y2, c;  // (x1,y1): 左上角, (x2,y2): 右下角, c: 增加值
15         cin >> x1 >> y1 >> x2 >> y2 >> c;
16         diff[x1][y1] += c;
17         diff[x2 + 1][y1] -= c;
18         diff[x1][y2 + 1] -= c;
19         diff[x2 + 1][y2 + 1] += c;
20     }
21     // 对差分数组做二维前缀和还原
22     for (int i = 1; i <= n; i++) {
23         for (int j = 1; j <= m; j++) {
24             diff[i][j] += diff[i-1][j] + diff[i][j-1] - diff[i-1][j-1];
25             cout << diff[i][j] << " ";
26         }
27         cout << endl;
28     }
29     return 0;
30 }

```

Listing 23: 二维差分——O(1) 子矩阵增减

4.5 前缀和——区间内特定值出现次数

适用场景：多次查询某个区间内某个值出现了多少次（或区间内满足某条件的元素个数）。对每种值分别建前缀和。

题目：给定一个只包含小写字母的字符串， q 次查询某区间内某字母的出现次数。

```

1  #include <iostream>

```

```
2 using namespace std;
3
4 const int MAXN = 100005;
5 int pre[MAXN][26];           // pre[i][c]: 前 i 个字符中字母 c 的出现次数
6
7 int main() {
8     string s;                 // s: 输入字符串
9     cin >> s;
10    int n = (int)s.size();     // n: 字符串长度
11    // 预处理: 统计每种字母的前缀出现次数
12    for (int i = 1; i <= n; i++) {
13        for (int c = 0; c < 26; c++) {
14            pre[i][c] = pre[i-1][c]; // 继承之前的前缀
15        }
16        pre[i][s[i-1] - 'a']++; // 当前字符累计加一
17    }
18    int q;                     // q: 查询次数
19    cin >> q;
20    while (q--) {
21        int L, R;              // L: 左边界, R: 右边界
22        char ch;               // ch: 查询的字母
23        cin >> L >> R >> ch;
24        int c = ch - 'a';      // c: 字母对应编号
25        // 原理: 区间内出现次数 = 前缀[R] - 前缀[L-1]
26        cout << pre[R][c] - pre[L-1][c] << endl;
27    }
28    return 0;
29 }
```

Listing 24: 前缀和——区间内字母出现次数

5 排序

排序是将无序数据按特定顺序排列的过程，是许多算法的基础预处理步骤。不同排序算法在时间复杂度、稳定性、空间复杂度方面各有优劣。

5.1 快速排序

适用场景：通用排序场景，平均 $O(n \log n)$ ，是实际应用中最高效的比较排序算法之一。不稳定排序。

题目：给定 n 个整数，将其从小到大排序。

```
1  #include <iostream>
2  using namespace std;
3
4  const int MAXN = 100005;
5  int a[MAXN];           // a[]: 待排序数组
6
7  // 原理: 选取基准 pivot, 将数组分为"小于 pivot"和"大于 pivot"两部分, 递归排序
8  // 分区函数: 返回基准元素的最终位置
9  int partition(int l, int r) {
10     int pivot = a[l + (r - l) / 2]; // pivot: 选取中间位置元素作为基准
11     int i = l - 1, j = r + 1;      // i: 左指针, j: 右指针
12     while (true) {
13         do { i++; } while (a[i] < pivot); // 从左找第一个 >= pivot 的元素
14         do { j--; } while (a[j] > pivot); // 从右找第一个 <= pivot 的元素
15         if (i >= j) return j;          // 指针相遇, 分区完成
16         swap(a[i], a[j]);              // 交换逆序对
17     }
18 }
19
20 void quickSort(int l, int r) {
21     if (l >= r) return;               // 区间长度 <= 1, 已有序
22     int p = partition(l, r);          // p: 分区点
23     quickSort(l, p);                  // 递归排序左半部分
24     quickSort(p + 1, r);              // 递归排序右半部分
25 }
26
27 int main() {
28     int n;                            // n: 数组长度
29     cin >> n;
30     for (int i = 0; i < n; i++) cin >> a[i];
```

```
31 quickSort(0, n - 1);
32 for (int i = 0; i < n; i++) cout << a[i] << " ";
33 cout << endl;
34 return 0;
35 }
```

Listing 25: 快速排序——分治 + 分区

5.2 归并排序

适用场景：通用排序，稳定排序，同时可顺便统计逆序对数量。适合需要稳定性的场景。

题目：给定 n 个整数，使用归并排序将其从小到大排序。

```
1  #include <iostream>
2  using namespace std;
3
4  const int MAXN = 100005;
5  int a[MAXN];           // a[]: 待排序数组
6  int tmp[MAXN];         // tmp[]: 归并时的临时数组
7
8  // 原理：将两个有序子数组合并为一个有序数组
9  void merge(int l, int mid, int r) {
10     int i = l, j = mid + 1, k = 1; // i: 左子数组指针, j: 右子数组指针, k: 临时数
        组指针
11     while (i <= mid && j <= r) {
12         if (a[i] <= a[j]) tmp[k++] = a[i++];
13         else tmp[k++] = a[j++];
14     }
15     while (i <= mid) tmp[k++] = a[i++]; // 左半有剩余
16     while (j <= r) tmp[k++] = a[j++]; // 右半有剩余
17     for (int p = l; p <= r; p++) a[p] = tmp[p]; // 写回原数组
18 }
19
20 void mergeSort(int l, int r) {
21     if (l >= r) return;
22     int mid = l + (r - l) / 2; // mid: 中点
23     mergeSort(l, mid);        // 递归排序左半
24     mergeSort(mid + 1, r);    // 递归排序右半
25     merge(l, mid, r);        // 合并两个有序部分
26 }
```

```
27
28 int main() {
29     int n;
30     cin >> n;
31     for (int i = 0; i < n; i++) cin >> a[i];
32     mergeSort(0, n - 1);
33     for (int i = 0; i < n; i++) cout << a[i] << " ";
34     cout << endl;
35     return 0;
36 }
```

Listing 26: 归并排序——分治 + 合并有序数组

5.3 归并排序求逆序对数量

适用场景：统计数组中 $i < j$ 且 $a_i > a_j$ 的数对个数。逆序对数量反映数组的有序程度，在排序问题、序列相似度分析中有重要应用。

题目：给定一个数组，求其中逆序对的总数。

```
1  #include <iostream>
2  using namespace std;
3
4  const int MAXN = 100005;
5  int a[MAXN];
6  int tmp[MAXN];
7  long long ans = 0;           // ans: 逆序对总数（可能超过 int 范围）
8
9  // 原理：在合并两个有序子数组时，每当从右子数组取元素，
10 // 说明左子数组中剩余的所有元素都大于当前元素，均构成逆序对
11 void merge(int l, int mid, int r) {
12     int i = l, j = mid + 1, k = l;
13     while (i <= mid && j <= r) {
14         if (a[i] <= a[j]) {
15             tmp[k++] = a[i++];
16         } else {
17             tmp[k++] = a[j++];
18             ans += mid - i + 1; // 左半剩余元素个数即为新增逆序对数
19         }
20     }
21     while (i <= mid) tmp[k++] = a[i++];
22     while (j <= r) tmp[k++] = a[j++];
```

```
23     for (int p = l; p <= r; p++) a[p] = tmp[p];
24 }
25
26 void mergeSort(int l, int r) {
27     if (l >= r) return;
28     int mid = l + (r - l) / 2;
29     mergeSort(l, mid);
30     mergeSort(mid + 1, r);
31     merge(l, mid, r);
32 }
33
34 int main() {
35     int n;
36     cin >> n;
37     for (int i = 0; i < n; i++) cin >> a[i];
38     mergeSort(0, n - 1);
39     cout << ans << endl;
40     return 0;
41 }
```

Listing 27: 归并排序——计数逆序对

5.4 快速选择（求第 k 小的数）

适用场景：在无序数组中查找第 k 小（或第 k 大）的元素，平均时间复杂度 $O(n)$ ，无需完全排序。适用于求中位数、Top K 问题。

题目：给定 n 个整数，求其中第 k 小的数（即从小到大排序后第 k 个位置的数）。

```
1  #include <iostream>
2  using namespace std;
3
4  const int MAXN = 100005;
5  int a[MAXN];
6
7  // 分区函数（同快速排序）
8  int partition(int l, int r) {
9      int pivot = a[l + (r - l) / 2];
10     int i = l - 1, j = r + 1;
11     while (true) {
12         do { i++; } while (a[i] < pivot);
13         do { j--; } while (a[j] > pivot);
```

```

14     if (i >= j) return j;
15     swap(a[i], a[j]);
16 }
17 }
18
19 // 原理：分区后只递归包含第 k 小的那一半，另一半舍弃
20 int quickSelect(int l, int r, int k) { // k: 查找第 k 小的数 (k 从 1 开始)
21     if (l == r) return a[l]; // 区间只剩一个数
22     int p = partition(l, r); // p: 分区点
23     int leftLen = p - l + 1; // leftLen: 左半部分的元素个数
24     if (k <= leftLen) {
25         return quickSelect(l, p, k); // 第 k 小在左半部分
26     } else {
27         return quickSelect(p + 1, r, k - leftLen); // 在右半部分找第 k-leftLen 小
28     }
29 }
30
31 int main() {
32     int n, k;
33     cin >> n >> k;
34     for (int i = 0; i < n; i++) cin >> a[i];
35     cout << quickSelect(0, n - 1, k) << endl;
36     return 0;
37 }

```

Listing 28: 快速选择——基于快排分区，只递归一边

5.5 计数排序

适用场景：数据范围较小（值域有限）的整数排序，时间复杂度 $O(n + m)$ ，其中 m 为值域大小。稳定排序。不适合数据范围很大或存在负数的场景。

题目：给定 n 个范围在 $[0, 10^5]$ 内的整数，将其从小到大排序。

```

1  #include <iostream>
2  using namespace std;
3
4  const int MAXV = 100005; // MAXV: 值域上限
5  int cnt[MAXV]; // cnt[x]: 数值 x 出现的次数
6
7  int main() {
8      int n; // n: 元素个数

```

```

9      cin >> n;
10     int maxv = 0;           // maxv: 实际数据中的最大值
11     // 原理: 统计每个数值出现的次数, 再按数值顺序输出
12     for (int i = 0; i < n; i++) {
13         int x;              // x: 当前读入的数值
14         cin >> x;
15         cnt[x]++;           // 计数
16         maxv = max(maxv, x);
17     }
18     for (int v = 0; v <= maxv; v++) {
19         while (cnt[v]--) {   // 输出 cnt[v] 次数值 v
20             cout << v << " ";
21         }
22     }
23     cout << endl;
24     return 0;
25 }

```

Listing 29: 计数排序——统计各值出现次数

5.6 基数排序

适用场景：对整数或定长字符串排序，通过从低位到高位逐位进行稳定排序（通常搭配计数排序），时间复杂度 $O(d(n+k))$ ， d 为位数。适合数值范围大但位数少的场景。

题目：给定 n 个非负整数，使用基数排序将其从小到大排序。

```

1  #include <iostream>
2  #include <cstring>
3  using namespace std;
4
5  const int MAXN = 100005;
6  int a[MAXN];           // a[]: 待排序数组
7  int b[MAXN];           // b[]: 辅助数组
8  int cnt[10];           // cnt[d]: 当前位数字 d 的出现次数 (基数为 10)
9
10 int main() {
11     int n;               // n: 元素个数
12     cin >> n;
13     int maxv = 0;        // maxv: 最大值, 用于确定位数
14     for (int i = 0; i < n; i++) {
15         cin >> a[i];

```

```
16     maxv = max(maxv, a[i]);
17 }
18 // 原理：从低位到高位，每次按当前位做一次稳定排序（计数排序）
19 // exp 表示当前处理的位权（1=个位，10=十位，100=百位...）
20 for (int exp = 1; maxv / exp > 0; exp *= 10) {
21     memset(cnt, 0, sizeof(cnt));
22     // 统计当前位各数字出现次数
23     for (int i = 0; i < n; i++) {
24         int digit = (a[i] / exp) % 10; // digit: a[i] 在当前位的数字
25         cnt[digit]++;
26     }
27     // 前缀和转换为位置索引（保证稳定性）
28     for (int d = 1; d < 10; d++) {
29         cnt[d] += cnt[d - 1];
30     }
31     // 从后往前遍历，确保稳定排序
32     for (int i = n - 1; i >= 0; i--) {
33         int digit = (a[i] / exp) % 10;
34         b[--cnt[digit]] = a[i]; // 放入对应位置
35     }
36     // 写回原数组
37     for (int i = 0; i < n; i++) a[i] = b[i];
38 }
39 for (int i = 0; i < n; i++) cout << a[i] << " ";
40 cout << endl;
41 return 0;
42 }
```

Listing 30: 基数排序——按位 LSD 排序（低位优先）

附录：算法对比总结

表 1: 各算法核心思想与适用条件对比

算法类别	核心思想	适用条件
枚举	遍历解空间中所有候选解	数据规模小 ($n \leq 20 \sim 30$), 解空间有限
贪心	每步选局部最优, 期望全局最优	具有贪心选择性质和最优子结构
二分查找	在单调区间上每次排除一半	数据有序或函数单调
二分答案	将最优化转为判定, 利用答案单调性	答案存在上下界且满足单调性
三分查找	在单峰/单谷函数上缩小区间	函数严格单峰或单谷
前缀和	预处理累加和, $O(1)$ 区间查询	静态数组, 需要频繁区间求和
差分	记录变化量, $O(1)$ 区间修改	批量区间修改后统一查询结果
快速排序	分治 + 分区, 平均 $O(n \log n)$	通用比较排序, 不稳定
归并排序	分治 + 合并, 稳定 $O(n \log n)$	需要稳定排序或统计逆序对
计数排序	统计出现次数, $O(n+m)$	值域小的整数排序
基数排序	按位稳定排序, $O(d(n+k))$	整数/定长串, 位数少